

Django 教學 11

CRUD與物件權限

第 17 章

從最小範例到專案實作與驗收

本份學習路線

先完成 [10_帳號登入與Session](#)。

- **第 17 章：編輯型 CBV 與授權**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [來源索引](#) · [實作手冊對照](#)

第 17 章

編輯型 CBV 與授權

完成留言或商品 CRUD，以作者、他人、匿名帳號驗證可操作範圍。

本章的操作環境與成果

先用 LearnBoard 理解表單與擁有權，再對照 LearnMart；兩個專案分別使用自己的資料庫。

完成成果： 完成留言或商品 CRUD，以作者、他人、匿名帳號驗證可操作範圍。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

17-1 Decorator 與 Mixin 的角色

Function View (checked-in `add_to_cart` 順序) :

```
@require_POST  
@login_required  
def add_to_cart(request, pk): ...
```

Class View :

```
class OrderListView(LoginRequiredMixin, ListView): ...
```

Decorator 包裝 function ; Mixin 透過多重繼承把行為組合進 class 。

17-2 三層授權模型

對保護操作依序問：

1. **登入**：是否有 authenticated user？
2. **角色**：是否是 seller／buyer？
3. **物件關係**：這個 product/order/cart item 是否屬於他？

只在 navbar 隱藏連結不是權限檢查；request 仍可直接送到 URL。

17-3 LoginRequiredMixin 要放在 generic view 前

```
class OrderListView(LoginRequiredMixin, ListView):  
    ...
```

多重繼承依 Method Resolution Order 尋找方法。把 access mixin 放前面，讓它能在 `dispatch()` 階段先檢查登入，再進入 ListView 行為。

17-4 LearnBoard：由伺服器指派留言作者

目前 `learnboard/board/views.py` 的新增留言流程：表單只提供 `content`，作者由伺服器從 `session` 指派。

```
class MessageCreateView(LoginRequiredMixin, CreateView):
    model = Message
    form_class = MessageForm
    template_name = "board/message_form.html"

    def form_valid(self, form):
        form.instance.author = self.request.user
        messages.success(self.request, "留言已發布。")
        return super().form_valid(form)
```

這和 LearnMart 的 `ProductCreateView` 指派 `seller`、checkout 指派 `buyer` / `total` 是同一條規則：**使用者可提交的欄位不等於伺服器可信的欄位。**

17-5 `get_absolute_url()` 與成功 redirect

目前 LearnMart | 節錄／重排 | `marketplace/models.py::Product`

```
def get_absolute_url(self):  
    return reverse(  
        "marketplace:product-detail",  
        kwargs={"pk": self.pk},  
    )
```

CreateView／UpdateView 未設定 `success_url` 時，可在 model instance 儲存後使用標準網址。

17-6 LearnBoard：編輯與刪除的擁有權防線

```
class MessageUpdateView(LoginRequiredMixin, UpdateView):  
    def get_queryset(self):  
        return Message.objects.filter(author=self.request.user)
```

```
class OwnerOrStaffMixin(LoginRequiredMixin, UserPassesTestMixin):  
    raise_exception = True  
  
    def test_func(self):  
        obj = self.get_object()  
        return obj.author_id == self.request.user.pk or self.request.user.is_staff
```

- 編輯：scoped queryset 讓非作者得到 404
- 刪除：`test_func()` 讓非作者得到 403，staff 可管理
- LearnMart 同樣先檢查 seller role，再以 `seller=self.request.user` 限制商品或訂單

17-6A 模板中的權限感知按鈕

LearnBoard 節錄／重排 | `templates/board/message_list.html` ; `post` 是目前留言

```
{% if post.author == user %}
    <a class="btn btn-outline-primary btn-sm"
      href="{% url 'board:update' post.pk %}">編輯</a>
    <a class="btn btn-outline-danger btn-sm"
      href="{% url 'board:delete' post.pk %}">刪除</a>
{% elif user.is_staff %}
    <a class="btn btn-outline-danger btn-sm"
      href="{% url 'board:delete' post.pk %}">刪除</a>
{% endif %}
```

作者看到編輯與刪除；非作者的 staff 只看到刪除；其他人不顯示按鈕。
刪除連結先前往確認頁，下一節才以 POST 執行刪除。

按鈕控制操作介面；真正授權由 View 的 `get_queryset()`／`test_func()` 檢查。

驗收時以非作者直接開啟編輯／刪除網址，仍應被擋下，不能只檢查按鈕是否隱藏。

17-7 LearnBoard：DeleteView 的確認與執行

現有程式節錄 | `board/views.py`；省略成功訊息 hook

```
from django.urls import reverse_lazy
from django.views.generic import DeleteView

class MessageDeleteView(OwnerOrStaffMixin, DeleteView):
    model = Message
    template_name = "board/message_confirm_delete.html"
    success_url = reverse_lazy("board:list")
```

GET 顯示確認頁，POST 才刪除；OwnerOrStaffMixin 先檢查作者或 staff。

`reverse_lazy` 延後解析刪除後的返回網址。

17-8 刪除驗收：畫面與資料都要確認

LearnBoard 的 URL：`/messages/<pk>/delete/`。

```
<form method="post">
  {% csrf_token %}
  <button type="submit">確認刪除</button>
</form>
```

1. 作者 GET 確認頁：留言仍存在。
2. 作者 POST：轉址成功，留言已刪除。
3. 非作者 POST：403，留言仍存在；staff 可管理。

此表單為教學最小片段；既有模板保留自己的版型與返回連結。

17-9 SellerRequiredMixin：角色層

目前 LearnMart | 逐字摘錄 | marketplace/views.py::SellerRequiredMixin

```
class SellerRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):  
    def test_func(self):  
        return self.request.user.isSeller
```

結果：

- 匿名者：導向 login
- 已登入 buyer：測試失敗，通常 403
- seller：繼續處理 request

17-10 角色正確仍不代表擁有物件

危險做法：

```
Product.objects.get(pk=self.kwargs["pk"])
```

安全範圍：

```
def get_queryset(self):  
    return Product.objects.filter(seller=self.request.user)
```

先縮小 queryset，再由 Generic View 用 URL `pk` 找物件。

17-11 CreateView：成功表單的 hook

```
class ProductCreateView(SellerRequiredMixin, CreateView):
    model = Product
    form_class = ProductForm
    template_name = "marketplace/form.html"

    def form_valid(self, form):
        form.instance.seller = self.request.user
        return super().form_valid(form)
```

先補 server-owned seller，再讓父類別儲存、取得 success URL 並 redirect。

17-12 UpdateView 應一開始就是 ownership-safe

目前 LearnMart | 節錄／重排 | marketplace/views.py::ProductUpdateView

```
class ProductUpdateView(SellerRequiredMixin, UpdateView):  
    model = Product  
    form_class = ProductForm  
  
    def get_queryset(self):  
        return Product.objects.filter(seller=self.request.user)
```

seller A 送 seller B 的 `pk` 時，查詢結果沒有該物件，因此回 404，而不是先抓出來再忘記檢查。

17-13 IDOR：問題不是 ID 可猜，而是存取未被限制

IDOR = Insecure Direct Object Reference。

`/orders/42/` 使用流水號不是漏洞本身；漏洞是任何登入者都能用 `pk=42` 取得別人的訂單。

安全方式：

```
def get_queryset(self):  
    return Order.objects.filter(buyer=self.request.user)
```

17-14 403、404、405 放在不同層

- 403：已識別使用者，但角色／政策拒絕
- 404：目前可見 queryset 中沒有這個物件
- 405：URL 存在，但不允許這個 HTTP method

這些 status code 讓 API／瀏覽器／測試能區分失敗原因；不要所有錯誤都 redirect 到首頁。

17-15 Generic editing view 如何處理檔案？

Django 的 `CreateView` / `UpdateView` 會在自己的 form-processing flow 中綁定 POST 與 FILES。

因此你仍需要：

- Model 有 `ImageField`
- HTML 有 `enctype="multipart/form-data"`
- settings 有 `MEDIA_ROOT` / `MEDIA_URL`
- 開發環境有 media URL routing

但通常不必在 `form_valid()` 再手動建立一個 form。

17-16 概念檢核

1. `.as_view()` 為何必要？
2. `get_queryset()` 與 `get_context_data()` 各負責什麼？
3. `super()` 在 context 與 form flow 中保留了什麼？
4. 為何 seller role 不足以保護 UpdateView？
5. 何時應回 403，何時用 scoped queryset 產生 404？

答案見配套實作手冊原第 3 章

17-17 LearnMart 實作

任務：用兩個 seller 驗證 ProductUpdateView 的三層授權。

驗收重點：

- 匿名者得到 login redirect
- buyer 得到 403
- seller A 可編輯自己的商品
- seller A 對 seller B 商品得到 404

LearnMart 測試資料、步驟與參考測試；LearnBoard 對應練習

第 17 章實作與離堂檢核

任務： 完成留言或商品 CRUD，以作者、他人、匿名帳號驗證可操作範圍。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： [LearnBoard 02 原第 4 章](#)；[LearnMart 02 原第 3 章](#)。手冊保留原章號，對照表列出本課位置。

本份完成與後續

下一份：[12_購物車與流程測試](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。
- 原始教材與合併去向見 [來源索引](#)。